Week 3

RH124 Chapter 5. Creating, Viewing and Editing Text Files

RH124 Chapter 6. Managing Local Users and Groups

1 Redirection

- In order to be able to apply filter commands and work with text streams, it is helpful to understand a few forms of redirection that can be used with most commands: pipelines, standard output redirection, error output redirection and input redirection.

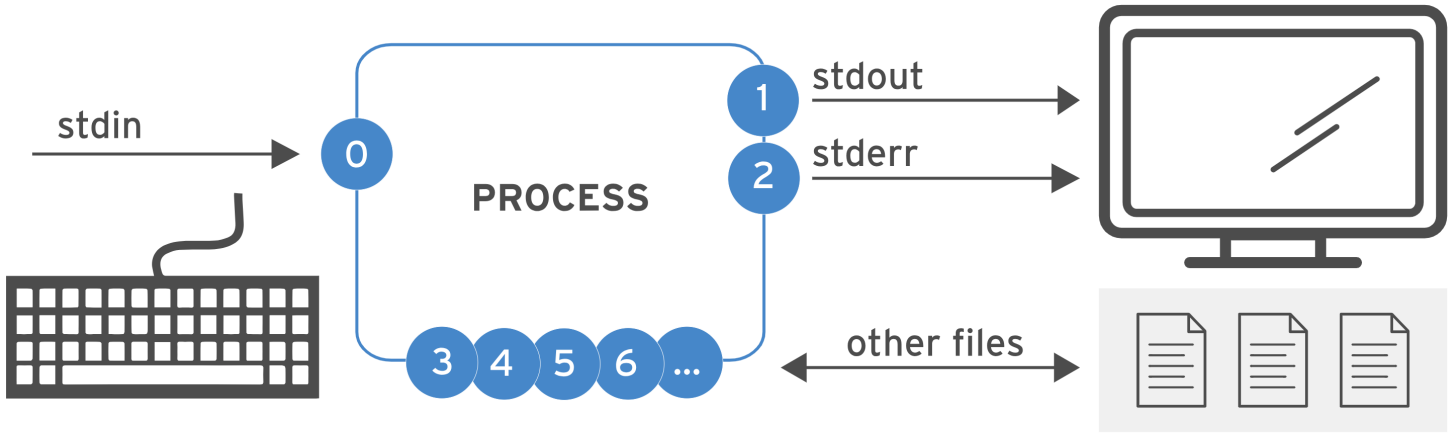


Figure 5.1: Process I/O channels (file descriptors)

Number	Channel Name	Description	Default connection	Usage
0	stdin	Standard input	Keyboard	Read only
1	stdout	Standard output	Terminal	Write only
2	stderr	Standard error	Terminal	Write only
3+	filename	Other files	None	Read, write or both

1.1 Standard Output (Channel 1)

- When a command executes without any errors, the output that is produced is known as *standard out*, also called *stdout* or *STDOUT*. By default, this output will be sent to the terminal where the command is being executed.
- It is possible to redirect standard out from a command so it will go to a file instead of the terminal. Standard output redirection is achieved by following a command with greater-than > character and a destination file. For example, the *ls ~* command will list the files in the home directory. To save a list of the files in the home directory, you must direct the output into a text file. To create the /tmp/home.txt file:

```
sysadmin@localhost:~$ ls ~ > /tmp/home.txt
```

If you now view the content of the /tmp/home.txt file using the cat command

```
sysadmin@localhost:~$ cat /tmp/home.txt
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
Videos
```

1.1 Standard Output (Channel 1) (Cont'd)

- Redirecting output using a single greater-than > character will create a new file, or *overwrite* the contents of an existing file with the same name. Redirecting standard output with two greater-than >> characters will also create a new file if it does not exist. The difference is that when using the >> characters, the output of the command will be *appended* to the end of a file if it does already exist. For example, to append to the file that was created by the previous command, execute the following:

```
sysadmin@localhost:~$ date
Sun Nov 24 23:36:43 UTC 2019
sysadmin@localhost:~$ date >> /tmp/home.txt
sysadmin@localhost:~$ cat /tmp/home.txt
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
Videos
Sun Nov 24 23:36:52 UTC 2019
```

1.1 Standard Output (Channel 1) (Cont'd)

- There is a number associated with the standard output file descriptor (the > character): the number 1 (one). However, since standard output is the most commonly redirected command output, the number can be omitted. Technically, the commands should be executed as shown below:

```
sysadmin@localhost:~$ ls 1> /tmp/ls.txt
sysadmin@localhost:~$ date 1>> /tmp/ls.txt
sysadmin@localhost:~$ cat /tmp/ls.txt
Desktop
Documents
Downloads
Music
Pictures
Public
Templates
Videos
Sun Nov 24 23:41:07 UTC 2019
```

1.2 Standard Error (Channel 2)

- When a command encounters an error, it will produce output that is known as *standard error*, also called `stderr` or `STDERR`. Like standard out, the standard error output is normally sent to the same terminal where the command is currently being executed. The number associated with the standard error file descriptor is `2` (two).

- If you tried to execute the ls /junk command, then the command would produce standard error messages because the /junk directory does not exist.

```
sysadmin@localhost:~$ ls /junk
ls: cannot access '/junk': No such file or directory
sysadmin@localhost:~$ ls /junk > output
ls: cannot access '/junk': No such file or directory
sysadmin@localhost:~$ ls /junk 2> /tmp/ls.err
sysadmin@localhost:~$ cat /tmp/ls.err
ls: cannot access '/junk': No such file or directory
```

- Because this output goes to standard error, the greater-than > character alone will not successfully redirect it, and the output of the command will still be sent to the terminal
- To redirect these error messages, you need to use correct file descriptor which is 2.

1.2 Standard Error (Channel 2) (Cont'd)

- Just like standard output, the use of a double >> character after the number 2 will append the error messages to a file or create the file if it does not exist.
- Some commands will produce both stdout and stderr output (eg, the find command below). You can redirect both the output to two separate files as follows:
-

```
sysadmin@localhost:~$ find /etc -name passwd
/etc/pam.d/passwd
/etc/passwd
find: '/etc/ssl/private': Permission denied
sysadmin@localhost:~$ find /etc -name passwd > /tmp/output.txt 2> /tmp/error.txt
sysadmin@localhost:~$ cat /tmp/output.txt
/etc/pam.d/passwd
/etc/passwd
sysadmin@localhost:~$ cat /tmp/error.txt
find: '/etc/ssl/private': Permission denied
```

1.2 Standard Error (Channel 2) (Cont'd)

- Sometimes it isn't useful to have error messages displayed in the terminal window or stored in a file. To discard these unwanted error messages, use the /dev/null file.
- /dev/null is like a trash can and anything that is sent to it is black-holed.

```
sysadmin@localhost:~$ find /etc -name passwd 2> /dev/null
/etc/pam.d/passwd
/etc/passwd
```

- If you wish to redirect all output to a single file, use either one of the following

```
sysadmin@localhost:~$ find /etc -name passwd &>> /tmp/finderr.txt
sysadmin@localhost:~$ cat /tmp/finderr.txt
/etc/pam.d/passwd
/etc/passwd
find: '/etc/ssl/private': Permission denied
sysadmin@localhost:~$ find /etc -name passwd > /tmp/finderr.txt 2>&1
```

1.3 Standard Input (Channel 0)

- *Standard in*, also called `stdin` or `STDIN`, normally comes from the keyboard with input provided by the user who runs the command. Although most commands are able to read input from files, there are some that expect the user to enter it using the keyboard.
- In some cases, it is useful to redirect standard input, so it comes from a file instead of the keyboard. A good example of when input redirection is desirable involves the *tr* command. The *tr* command *translates* characters by reading data from standard input; translating one set of characters to another set of characters and then writing the changed text to standard output.
- For example, the following *tr* command would take input from a user (via the keyboard) to perform a translation of all lowercase characters to uppercase characters. Execute the following command, type some text, and press **Enter** to see the translation:

```
sysadmin@localhost:~$ tr 'a-z' 'A-Z'
hello
HELLO
```

1.3 Standard Input (Channel 0)

- The *tr* command doesn't stop reading from standard input unless it's terminated or receives an "End of Transmission" character. This can be accomplished by typing **Ctrl+D**.
- The *tr* command won't accept a file name as an argument on the command line. To perform a translation using a file as input, utilize input redirection. To use input redirection, type the command with its options and arguments followed by the less-than < character and a path to a file to use for input. See example on the right.

- The contents of the animals.txt file is translated to uppercase characters.

```
sysadmin@localhost:~$ cat Documents/animals.txt
1 retriever
2 badger
3 bat
4 wolf
5 eagle
sysadmin@localhost:~$ tr 'a-z' 'A-Z' < Documents/animals.txt
1 RETRIEVER
2 BADGER
3 BAT
4 WOLF
5 EAGLE
```

1.4 Command Pipelines

- Command pipelines are often used to make effective use of filter commands. In a command pipeline, the output of one command is sent to another command as input. In Linux and most operating systems, the vertical bar or pipe | character is used between two commands to represent a command pipeline.
- For example, imagine that the output of the *ls /etc* command is very large. To send this output to the *grep* command to filter out the portion of the output that you wish to see. In the following example, the grep command is used to select only file starting with the letter a, b, c, d, e and f and ending with .conf extension

```
sysadmin@localhost:~$ ls /etc | grep -E ^[a-f].*.conf
adduser.conf
ca-certificates.conf
debconf.conf
deluser.conf
fuse.conf
```

1.4.1 xargs Command

- The *xargs* command is most useful when it is called in a pipe. In the next example, four files will be created using the *touch* command. The files will be named `1a`, `1b`, `1c`, and `1d` based on the output of the *echo* command.

```
sysadmin@localhost:~$ echo '1a 1b 1c 1d' | xargs touch
sysadmin@localhost:~$ ls
1a  1c  Desktop    Downloads  Pictures  Templates
1b  1d  Documents  Music      Public    Videos
```

- These files can be removed just as easily by changing the *touch* command to the *rm* command.

```
sysadmin@localhost:~$ echo '1a 1b 1c 1d' | xargs rm
sysadmin@localhost:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos
```

1.4.1 xargs Command (Cont'd)

- In the next example, the *find* command output is used as an argument to the *du* disk usage command using the *xargs* command. The first command preceding the pipe finds all file names that match the D* pattern. The output of the *find* command is then passed to the *du* command as an argument to display the disk usage of the files.

```
sysadmin@localhost:~$ find ~ -maxdepth 1 -name 'D*' | xargs du -sh
1.2M    /home/sysadmin/Documents
4.0K    /home/sysadmin/Downloads
4.0K    /home/sysadmin/Desktop
```

- The *xargs* command has many, many more options that can be studied using the corresponding man page. The example on the right shows a more complex use of the *xargs* command

```
sysadmin@localhost:~$ df --local -P | awk '(NR!=1) {print $6}' | xargs -I '{}' f
ind '{}' -type d -name 'python?.?' 2> /dev/null
/usr/share/doc/python3.6
/usr/local/lib/python3.6
/usr/lib/python3.7
/usr/lib/python3.6
/usr/lib/python2.7
/etc/python3.6
```

1.5 Redirection Summary

Usage	Explanation
> file	Redirect stdout to overwrite a file
>> file	Redirect stdout to append to a file
2> file	Redirect stderr to overwrite a file
2> /dev/null	Discard stderr error messages by redirecting them to /dev/null
> file 2>&1 &> file	Redirect stdout and stderr to overwrite the same file
>> file 2>&1 &>> file	Redirect stdout and stderr to append to the same file

2 Vim

- The main / native editor for Linux and UNIX is the `vi` program. While there are numerous editors available for Linux that range from the tiny editor `nano` to the massive `emacs` editor, there are several advantages to the `vi` editor.
  - The `vi` editor is available on every Linux and UNIX distribution in the world.
  - The `vi` editor can be executed both in a CLI and a GUI while graphical editors like `gedit` and `kedit` can only be executed from a GUI, which servers won't always have running.
  - The `vi` editor have been around since 1970s. Although new features have been added over the years, the core functions have been around for decades and any system administrators conversant with the `vi` editor can apply their knowledge to all UNIX-based systems since the 1970s.
- Most Linux systems today don't include the original version of `vi` found in the 1970s UNIX systems, but instead include an improved version of it known as `vim`, for `vi` improved.

```
[sysadmin@centos ~]$ which vi
alias vi='vim'
        /usr/bin/vim
[sysadmin@centos ~]$
```

2 Vim (Cont'd)

- `Vim` works just like `vi` but has additional improved features. Eg, in `vim` you can use the arrow keys for navigation which is absent in `vi`. `Vi` uses the `j` key for down, `k` key for up, `h` key for left and `l` key for right. This may seem weird at first but these are keys (except for letter `h`) which you will place the fingers of your right hand on the qwerty key board.
- Nano is a light-weight editor with a helpful menu at the bottom of the editor screen. It is useful for users who have not (or didn't bother to) memorised all the `vi` commands.
- To get started with `vi`, simply type `vi newfile` as follows

```
[sysadmin@centos ~]$ vi newfile
```

2.1 Command Mode Movement

- There are three modes used in the vi editor: command mode ,insert mode and the ex mode.
- The vi program will always start in command mode. Command mode is used to type commands, such as those used to move around a document, manipulate text, and access the other two modes. To return to command mode at any time, press the Esc key.
- Once some text has been added into a document, to perform actions like moving the cursor, the Esc key needs to be pressed first to return to command mode. This seems like a lot of work, but remember that vi works in a terminal environment where a mouse is useless.
- Cursor and text manipulation commands in vi have two aspects: a motion and an optional number prefix, which indicates how many times to repeat that motion. The general format is as depicted on the right.

```
[count] motion
```

Motion	Result
h	Left one character
j	Down one line
k	Up one line
l	Right one character
w	One word forward
b	One word back
^	Beginning of the line
$	End of the line

- The motion keys in the previous slide can be prefixed with a number to indicate how many times to perform the movement. For example, 5w will move cursor forward 5 words.

2.2 Command Mode Actions

- The standard convention for editing content with word processors is to use copy, cut and paste. The vi program does not use these; instead, it uses the three commands listed in the table on the right.

Standard	Vi	Meaning
cut	d	delete
copy	y	yank
paste	P | p	put

- Delete removes the indicated text from the page and saves it into the buffer, the buffer is the equivalent of the “clipboard” used in Windows. The table on the right provides some common usage examples.

Action	Result
dd	Delete the current line
3dd	Delete the next three lines
dw	Delete the current word
d3w	Delete the next three words
d4h	Delete the four characters to the left

2.2 Command Mode Actions (Cont'd)

- Change is very similar to delete, where the text is removed and saved into the buffer. However, when using change, the program is switched to insert mode to allow immediate change to the text. The table on the right provides some common usage examples.

- Yank places the content into the buffer without deleting it. The table on the right provides some common usage example.

Action	Result
cc	Change the current line
5cc	Change the next 5 lines
cw	Change the current word
c3w	Change the next three words
c5h	Change the five characters to the left

Action	Result
yy	Change the current line
5yy	Change the next 5 lines
yw	Change the current word
y$	Change to the end of the line
y5l	Change the five characters to the right

2.2 Command Mode Actions (Cont'd)

- Put places the text saved in the buffer either before or after the cursor position. Notice that these are the only two options as put does not use the motions like the previous action commands.
- Searching in vi. To search forward, type the / key in command mode followed by the search pattern. To search backwards, type the ? in command mode followed by the search pattern.

Action	Result
p	Put (paste) after the cursor
P	Put before cursor

Action	Result
/term	Search forwards for term
?term	Search backward for term

2.3 Insert Mode

- Insert mode is used to add text to the document. There are a few ways to enter insert mode from command mode, each differing by where the text insertion will begin. The following table covers the most common. After using one of these commands, you may enter text. To return to command mode, use the Esc key.

Input	Purpose
a	Enter insert mode right after the cursor
A	Enter insert mode at the end of the line
i	Enter insert mode right before the cursor
I	Enter insert mode at the beginning of the line
o	Enter insert mode on a blank line after the cursor
O	Enter insert mode on a blank line before the cursor

2.4 Ex Mode

- When the ex mode of the vi editor is being used, it is possible to view or change settings, as well as carry out file-related commands like opening, saving, or aborting changes to a file. In order to get to the ex mode, type a colon : character in command mode. The following table lists some common actions performed in ex mode:

Input	Purpose
:w	Write the current file to the filesystem
:w filename	Save a copy of the current file as filename
:w!	Force writing to the current file
:1	Go to line number 1 or whatever line number is given
:e filename	Open filename
:q	Quit if no changes made to file
:q!	Quit without saving changes to file
:wq	Write the current file to the file system and quit the vi editor

3. Shell Variables

- A *variable* is a name or identifier that can be assigned a value. The shell and other commands read the values of these variables, which can result in altered behavior depending on the contents (value) of the variable.
- Variables typically hold a single value. Some may hold multiple values separated by spaces, semi-colons, colons or commas etc.
- Variables are assigned values by typing the name of the variable immediately (without spaces) followed by the equal sign ("=" character) and then the value. For example,

```
myvariable="Bob Smith"
```

- Variable names should start with a letter or underscore and should only contain letters, numbers and the underscore character. Variable names are also case sensitive.
- When assigning values that includes special characters to variables, you should use the double quotes.

3. Shell Variables (Cont'd)

- The Bash shell and many commands make extensive use of variables. One of the main uses of variables is to provide a means to configure various preferences for each user. The Bash shell and commands can behave in different ways, based on the value of variables. For the shell, variables can affect what the prompt displays, the directories the shell will search for commands to execute and much more.

3.1 Local and Environment Variables

- A *local variable* is only available to the shell in which it was created. An *environment variable* is available to the shell in which it was created, and it is passed into all other commands/programs started by the shell.
- In the example below, a local variable is created, and the echo command is used to display its value:

```
sysadmin@localhost:~$ name="judy"
sysadmin@localhost:~$ echo $name
judy
```

- An environment variable can be created by using the export command

```
sysadmin@localhost:~$ export JOB=engineer
sysadmin@localhost:~$ echo $JOB
engineer
```

3.1 Local and Environment Variables (Cont'd)

- You can compare between the local and environment variables by opening a new shell with the *bash* command and displaying the values of the variables again. Only the environment variable is recognized by the new shell.

```
sysadmin@localhost:~$ bash
sysadmin@localhost:~$ echo $name

sysadmin@localhost:~$ echo $JOB
engineer
```

- Type exit to exit the new shell and return to the previous shell. Attempt to display the values of both the name and JOB variables again.

```
sysadmin@localhost:~$ exit
exit
sysadmin@localhost:~$ echo $name
judy
sysadmin@localhost:~$ echo $JOB
engineer
```

3.1 Local and Environment Variables (Cont'd)

- You can convert a local variable to an environment variable by running the *export* command

```
sysadmin@localhost:~$ export name
sysadmin@localhost:~$ bash
sysadmin@localhost:~$ echo $name
judy
sysadmin@localhost:~$ echo $JOB
engineer
```

3.2 Displaying Variables

- You can display all the variables (local and environment) and the values assigned to them by using the *set* command (in the example below, we have redirected the output of the set command to the *head* command to just display the first 10 lines of the *set* command output. Redirection is out of the scope of this lesson).

```
sysadmin@localhost:~$ set | head
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglob:extquote
:force_fignore:histappend:interactive_comments:progcomp:promptvars:sourcepath
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
BASH_CMDS=()
BASH_COMPLETION_VERSINFO=([0]="2" [1]="8")
BASH_LINENO=()
BASH_SOURCE=()
BASH_VERSINFO=([0]="4" [1]="4" [2]="19" [3]="1" [4]="release" [5]="x86_64-pc-lin
ux-gnu")
```

- You can also use the *env* command to only display environment variables or the *echo* command to display only specific variables (eg, echo $PATH)

3.3 Unsetting Variables

- You can use the *unset* command to delete a variable

```
sysadmin@localhost:~$ unset JOB
sysadmin@localhost:~$ echo $JOB
```

- Warning – unsetting critical system variables like the PATH variable can cause the system to malfunction

3.4 PATH Variables

- The PATH variable is one of the most critical environment variables for the shell. It contains a list of directories that are used to search for commands entered by the user. When the user types a command and then presses the Enter key, the PATH directories are searched for an executable file that matches the command name. Processing works through the list of directories from left to right and the first executable found that matches the command typed will be executed by the shell.
- Use the *echo* command to display the current value assigned to the PATH variable.

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/
usr/bin:/sbin:/bin:/usr/games:/usr/local/games
```

- The next slide list the purpose of some of the directories displayed in the output of the previous command.

3.4 PATH Variables (Cont'd)

Directory in PATH	Contents
/home/<current user>/bin	A directory for the current user to place programs. Typically used by users who create their own scripts
/usr/local/sbin	Normally empty, but may have administrative commands that have been compiled from local sources
/usr/local/bin	Normally empty, but may have commands that have been compiled from local sources
/usr/sbin	Contains the majority of the administrative command files
/usr/bin	Contains the majority of the commands that are available for regular users to execute
/sbin	Contains the essential administrative commands
/bin	Contains the most fundamental commands that are essential for the operating system to function

3.4 PATH Variables (Cont'd)

- To execute commands that reside in directories NOT listed in the PATH variable, the following options exist:
  - the command may be executed by typing the absolute (commonly called FULL) path or relative path to the command
  - update the PATH variable to include the directory where the command resides
  - copy the command to one of the directories in the PATH variable

Given an executable script my.sh, an example of the absolute and relative executable is shown on the right:

```
sysadmin@localhost:~$ pwd
/home/sysadmin
sysadmin@localhost:~$ ls
Desktop    Downloads  Pictures  Templates  my.sh
Documents  Music      Public    Videos
sysadmin@localhost:~$ #absolute path execution
sysadmin@localhost:~$ /home/sysadmin/my.sh
Hello World!
sysadmin@localhost:~$ #relative path execution
sysadmin@localhost:~$ ./my.sh
Hello World!
```

3.4 PATH Variables (Cont'd)

- To modify (add additional directories) the PATH variable, you need to use the following command

```
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games
sysadmin@localhost:~$ PATH=$PATH:/etc/bin
sysadmin@localhost:~$ echo $PATH
/home/sysadmin/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/etc/bin
```

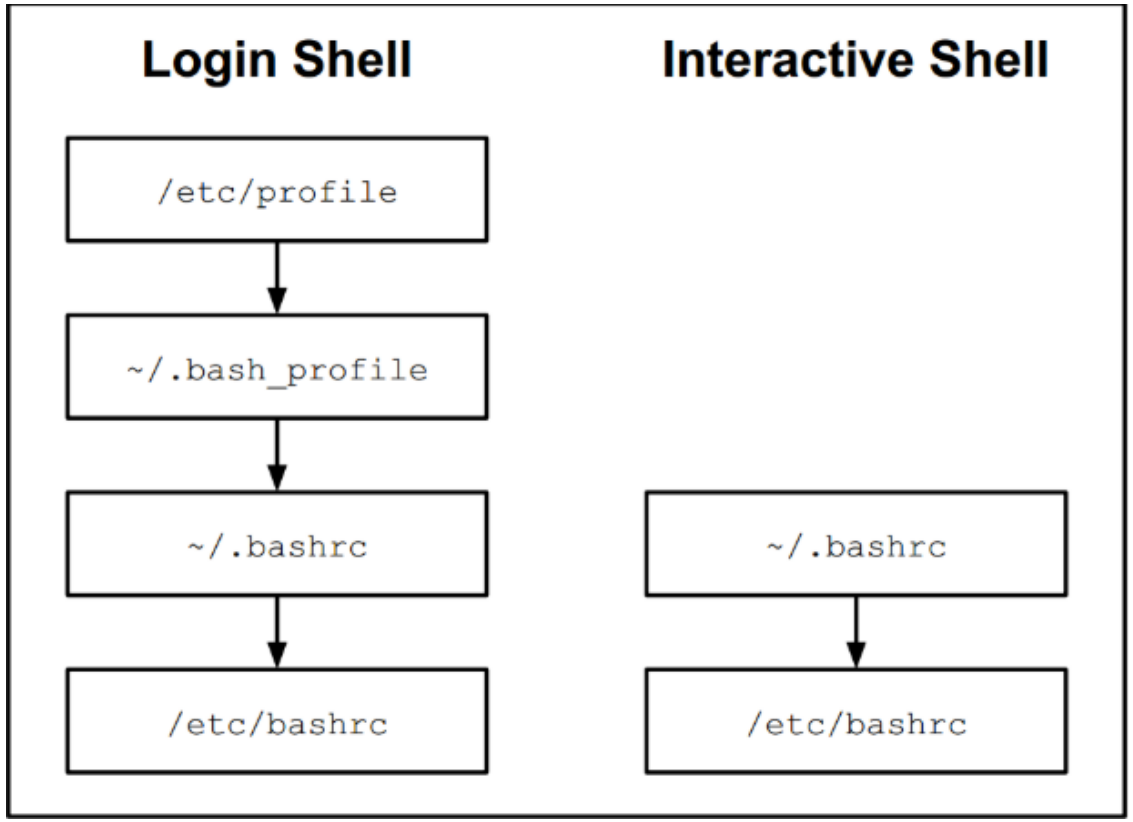
- If you modify the PATH variable in the wrong way as below, all executable or script files outside of the /etc/bin folder can only be executed if you type the full path name

```
sysadmin@localhost:~$ PATH=/etc/bin
```

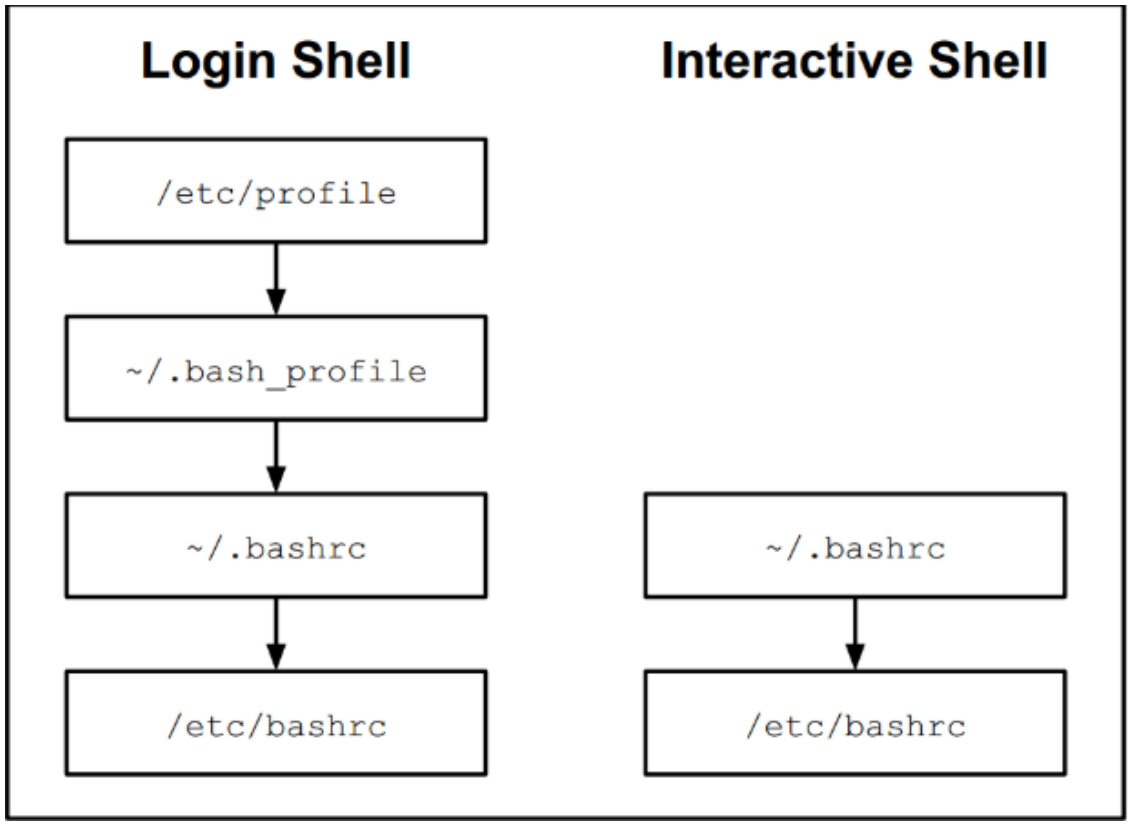
- To restore the original value of the PATH variable, type *exit*. This is only possible if you have not run *export PATH*.

3.5 Setting Variables Automatically

- When a user opens a new shell, either during login or when they run a terminal that starts a shell, the shell is customized by files called *initialization (or configuration) files*. These initialization files set the value of variables, create aliases and functions, and execute other commands that are useful in starting the shell.
- There are two types of initialization files: *global initialization files* that affect all users on the system and *local initialization files* that are specific to an individual user.
- Global configuration files are located in /etc directory. Local configuration files are stored in the user's home directory.
- The following diagram illustrates the different files that are started with a typical login shell versus an interactive (non-login) shell:



3.5 Setting Variables Automatically (Cont'd)



- When Bash is started as a login shell, */etc/profile* is first executed (also runs all *.sh* files in */etc/profile.d* directory) followed by *~/.bash_profile*. *~/.bash_profile* will also call *~/.bashrc* which in turn executes */etc/bashrc*.
- When Bash is started as an interactive (non-login) shell (eg, starting terminal application in GUI, no login is required), it executes the *~/.bashrc* file which may also execute the */etc/bashrc* file if it exists.

3.5 Setting Variables Automatically (Cont'd)

- The following table illustrates the purpose of each of these files, providing examples of what commands you might place in each file:

File	Purpose
/etc/profile	This file can only be modified by the administrator and will be executed by every user who logs in. Administrators use this file to create key environment variables, display messages to users as they log in, and set key system values.
~/.bash_profile	Each user has their own .bash_profile file in their home directory. The purpose of this file is the same as the /etc/profile file, but having this file allows a user to customize the shell to their own tastes. This file is typically used to create customized environment variables.
~/.bashrc	Each user has their own .bashrc file in their home directory. The purpose of this file is to generate items that need to be created for each shell, such as local variables and aliases.
/etc/bashrc	This file may affect every user on the system. Only the administrator can modify this file. Like the .bashrc file, the purpose of this file is to generate items that need to be created for each shell, such as local variables and aliases.

4. Managing Local Users and Groups

- System administration is a broad topic but generally involves the setup and maintenance of servers and user computers on a network. Daily tasks can include creating and maintaining accounts, installing software, ensuring system security, backing up data, and solving computer and network problems. This lesson focuses on creating and maintaining user and group accounts, password security, automating administration and maintenance tasks, and localization features (i.e. language, timezone, etc.) for systems anywhere in the world.
- Linux is a multiuser operating system, allowing multiple users to access the system simultaneously. User and group accounts facilitate this by controlling the access allowed for different types of users.

4.1User and System Account Files

- The /etc/passwd file contains the details of special system accounts (ie, bin, lp, nobody, etc) used to run services, and user accounts on the system. Without an entry in the /etc/passwd file, users are unable to log in to the system. This file has read access for all users, but write access is limited to the root user. This file can be viewed with any of the file viewing commands such as cat, more, less, head or tail.

```
sysadmin@localhost:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
sshd:x:103:65534::/var/run/sshd:/usr/sbin/nologin
operator:x:1000:37::/root:
sysadmin:x:1001:1001:System Administrator,,,,:/home/sysadmin:/bin/bash
```

4.1 User and System Account Files (Cont'd)

- Each row in this file describes one user account record. Each passwd record contains the following seven fields separated by colons:

```
loginID:x:UID:GID:comment:home_directory:login_shell
```

Field	Example	Significance
Login ID	`sysadmin`	Login name of the user
Password	`x`	'x' indicates that the encrypted password has been stored in `/etc/shadow` file
UID	`1001`	User ID assigned by the system (This is a non-zero value for regular users since root always has UID 0)
GID	`1001`	Primary Group ID
GECOS	`System Administrator,,,,`	Comments, if any, are specified in this field. Note that commas are not delimiters, only colons
Home Directory	`/home/sysadmin`	Absolute path of the default home directory of the user
Shell	`/bin/bash`	Absolute path of the user's login shell

4.1 User and System Account Files (Cont'd)

- The /etc/passwd was adequate in the early days of Unix, but with increased security concerns, the /etc/shadow file was introduced. The /etc/shadow file contains the encrypted password of the user and some parameters related to password security. In addition to providing the administrator with the ability to set password expiration fields, the /etc/shadow file also increases security because, unlike the /etc/passwd file, only the root user can view the /etc/shadow file. This means that regular users can't even see the encrypted password.
- For every user entry in the /etc/passwd file, an entry exists in the /etc/shadow file. The /etc/passwd file only contains an * or x in the password field as a placeholder for the user's encrypted password stored in the /etc/shadow file.

```
sysadmin@localhost:~$ sudo tail /etc/shadow
list:*:16862:0:99999:7:::
irc:*:16862:0:99999:7:::
gnats:*:16862:0:99999:7:::
nobody:*:16862:0:99999:7:::
libuuid:!:16862:0:99999:7:::
syslog:*:16862:0:99999:7:::
bind:*:16874:0:99999:7:::
sshd:*:16874:0:99999:7:::
operator:!:16874:0:99999:7:::
sysadmin:$6$ZU5DV8gO$qtEUl2mAJJE.aNbevunRfHuhyqaXBZcyHoys6diRmikuIl1xmgnZCMoBjdg
GvIsyXnbUZrQYogRlor9q95Uyz.:16874:0:99999:7:::
```

4.1 User and System Account Files (Cont'd)

- Each record in /etc/shadow file contains the following eight fields, separated by colons:

```
loginID:password:lastchg:min:max:warn:inactive:expire
```

Field	Example	Significance
Username	`sysadmin`	Login name of the user

Password	`$6$trs.W5GR$sgPBwHSIR6Zfcw7oe05mr`
	`mtR3slZQRDur02NZ71OmZ7Uaz0Xrl0ME`
	`7tdMClY9rrX5Jlh1zASnXFCuYFgNHzU/0`
		Encrypted password.
		Note: An empty field indicates no password is required to login. An asterisk `*` or exclamation mark `!` indicates an inaccessible account (typically system) with no password. A password field that starts with `!` followed by the encrypted password, indicates the password is locked.

Last password change	`16413`	Number of days between January 1, 1970, and the last password change
Minimum	`0`	Minimum number of days that the current password can be changed by the user; a value of 0 means "no minimum password age"

Field	Example	Significance
Maximum	`99999`	Maximum number of days remaining for the password to expire; a value of 99999 means "no maximum password age"
Warn	`7`	Number of days prior to password expiry that the user is warned
Inactive	`30`	Number of days after password expiry that the user account remains active
Expire	`17000`	Date indicating when the password is deleted and user name will become inactive
Reserved field		Reserved for future use

4.2 Group Accounts

- Users are organized in groups to facilitate management, monitoring and control of users and resources. Below are two examples of using a group:
  - The root user can create an admin group, which can be configured to allow group members the rights to execute administration-based tasks.
  - When a server is being shared by multiple teams in an organization, a separate group can be created for each team (i.e. marketing, sales, engineering, etc.). Their shared resources, such as files and directories, will be accessed securely by team members only.
- Every user is associated with at least one group known as their primary group. Recall that the user's primary group is stored in the fourth field of the /etc/passwd file. Any additional group membership will be indicated in the /etc/group file, which is used to store group account information.

4.2 /etc/group File

- The /etc/group contains the list of groups created in the system. It is automatically populated with system groups such as daemon, bin, sys etc. In addition, any user private group created will also be added automatically to the /etc/group file when the user account is created.
- Each record in the /etc/group file has the following format.

```
group_name:password:GID:group_list
```

Field	Example	Significance
Group Name	`marketing`	Name of the group
Password		This field is blank for most of the groups Traditionally, it can store an encrypted password for privileged groups, although most modern distributions of Linux stores group passwords in the `/etc/gshadow` file An `x` in this field indicates there might be a group password in the `/etc/gshadow` file
GID	`1002`	The Group ID
Group List	`Madeleine,Colin,Condoleezza`	List of the user ids who are members of this group

4.2 /etc/group File (Cont'd)

- A user account can only belong to one primary group but also belong to one or more secondary groups. Use the id <username> command to show the primary group and secondary groups of the user:

```
root@localhost:~# id sysadmin
uid=1001(sysadmin) gid=1001(sysadmin) groups=1001(sysadmin),4(adm),27(sudo)
```

- If for some reason, you would like a user account to use a particular secondary group as its current primary group, you can use the newgrp command:

```
sysadmin@localhost:~$ id
uid=1001(sysadmin) gid=1001(sysadmin) groups=1001(sysadmin),4(adm),27(sudo)
sysadmin@localhost:~$ newgrp adm
sysadmin@localhost:~$ id
uid=1001(sysadmin) gid=4(adm) groups=1001(sysadmin),4(adm),27(sudo)
```

- Note that in the example above, the primary of the sysadmin user account was originally sysadmin. After executing the newgrp command, a new shell is spawn and the adm group is used as the primary group. You can exit the new shell by typing exit.

```
sysadmin@localhost:~$ exit
exit
sysadmin@localhost:~$ id
uid=1001(sysadmin) gid=1001(sysadmin) groups=1001(sysadmin),4(adm),27(sudo)
```

4.3.1 Switching Users (su)

- The *su* command allows you to run a shell as a different user and is most frequently used by a normal user account to switch to the root user.

```
su [OPTIONS] [USERNAME]
```

- When switching users, utilizing the login shell option (*- , -l or --login*) is recommended as the login shell fully configures the new shell with the settings of the new user, ensuring any commands executed run correctly. If the username argument is omitted, it is defaulted to the root user. Hence *su -* is equivalent to *su - root* is equivalent to *su -l root* is equivalent to *su --login* root.

- After pressing enter at the *su* command, the user must provide the password of the username he/she is attempting to switch to. Example, if the command is *su -*, then the user must provide the root user's password. Type *exit* to return to the original user's shell and logout from the switched user account.

```
sysadmin@localhost:~$ su -
Password:
root@localhost:~# id
uid=0(root) gid=0(root) groups=0(root)
root@localhost:~#
root@localhost:~# exit
logout
sysadmin@localhost:~$
```

4.3.2 Switching Users (sudo)

- The sudo command allows the root user to delegate privileged operations to user accounts. This is the preferred way (when compared to using the su command) to allow normal user accounts to carry out privileged command.

```
sudo [OPTIONS] COMMAND
```

- When using the sudo command to execute a command as the root user, the command prompt for the user's own password and not that of the root user. This security feature prevents abuse of the users' knowledge of the root password in the case of the su command.

```
sysadmin@localhost:~$ head /etc/shadow
head: cannot open '/etc/shadow' for reading: Permission denied
sysadmin@localhost:~$ sudo head /etc/shadow
[sudo] password for sysadmin:
root:$6$gWn1b01z$/awsehiCL9PPBmlYgEbtpeSXkafiLZPcugO5VjU.R5bjhGlg8IoTOLVmkBoXlq0
bOBUQ0lCkVHmg2ucnoHpsY0:18010:0:99999:7:::
daemon:*:17962:0:99999:7:::
bin:*:17962:0:99999:7:::
sys:*:17962:0:99999:7:::
sync:*:17962:0:99999:7:::
games:*:17962:0:99999:7:::
man:*:17962:0:99999:7:::
lp:*:17962:0:99999:7:::
mail:*:17962:0:99999:7:::
news:*:17962:0:99999:7:::
```

4.3.2 Switching Users (sudo) (Cont'd)

- One of the key security benefit of using sudo over su is that sudo allows fine grained control of the user accounts that can execute sudo and the commands they can execute sudo with.

```
[student@sads-lab ~]$ sudo ls /root

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for student:
student is not in the sudoers file.  This incident will be reported.
[student@sads-lab ~]$
```

- The root user can restrict the use of sudo to specific user accounts/groups through the /etc/sudoers file or any files within the /etc/sudoers.d folder

```
## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)       ALL
```

- The *%wheel* string is the user or group that the rule applies to. The *%* symbol before the *wheel* word specifies a group.
- The *ALL=(ALL)* command specifies that on any host with this file (the first ALL), users in the *wheel* group can run commands as any other user account on the system (the second ALL).
- The final ALL command specifies that users in the *wheel* group can run any command.

4.4 Special Purpose System Accounts

- As introduced earlier, Linux systems come with special purpose system accounts for root, ssh, ftp, mail, news and other system users. These accounts are defined in /etc/passwd and are not accessible to regular users. They are used for running services, which are closely linked with a specific set of programs or applications. System accounts will be managed by the root user or admin group users only.

- The /etc/login.defs file contains a setting that allows the administrator to specify which number the regular user accounts start from:

```
sysadmin@localhost:~$ egrep "^UID|^GID" /etc/login.defs
UID_MIN                  1000
UID_MAX                 60000
GID_MIN                  1000
GID_MAX                 60000
```

Parameter	Type	Meaning
UID_MAX	number	Maximum number of the UID range
UID_MIN	number	Minimum number of the UID range
GID_MAX	number	Maximum number of the GID range
GID_MIN	number	Minimum number of the GID range

- The UID_MIN setting means the UID assigned to the first user account created on the system. This is then automatically increased by 1 for each additional user created. This also applies to group IDs using the GID_MIN setting in the /etc/login.defs file.

4.5.1 Managing User Accounts (Create)

- Creating a new user is a two-step process:
  1. Create the new account
  2. Set a password for the new account
- The useradd command is used to create a new user account. To add a new user named test_user, run the following command as root:

```
sysadmin@localhost:~$ sudo useradd test_user
```

- Note that when successfully executed, the command produces no output. To verify the command was executed successfully, use the tail -1 command to display the last line of the /etc/passwd and /etc/shadow files:

```
sysadmin@localhost:~$ sudo tail -1 /etc/passwd
test_user:x:1002:1002::/home/test_user:
sysadmin@localhost:~$ sudo tail -1 /etc/shadow
test_user:!:18242:0:99999:7:::
```

4.5.1 Managing User Accounts (Create) (Cont'd)

- Notice the password field in the /etc/shadow for the test_user account is set to !. This means the account is locked until a password has been set for this new account using the passwd command.

- To view the account status of a user, use the -S option of the passwd command. On the right is a capture of the man passwd output of the -S option.

```
sysadmin@localhost:~$ sudo passwd -S test_user
test_user L 12/12/2019 0 99999 7 -1
```

```
-S, --status
    Display account status information. The status information consists
    of 7 fields. The first field is the user's login name. The second
    field indicates if the user account has a locked password (L), has
    no password (NP), or has a usable password (P). The third field
    gives the date of the last password change. The next four fields
    are the minimum age, maximum age, warning period, and inactivity
    period for the password. These ages are expressed in days.
```

- Set the password of the account as shown on the right. Notice that the status of the account has been changed to P immediately after a password was set for the account.

```
sysadmin@localhost:~$ sudo passwd test_user
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
sysadmin@localhost:~$ sudo passwd -S test_user
test_user P 12/12/2019 0 99999 7 -1
```

4.5.1 Managing User Accounts (Create) (Cont'd)

- The execution of the useradd test_user command previously creates the new account using default options. The main issues of executing the useradd command with no options is that the test_user account is created without the home directory and the shell is set to the value currently configured in /etc/default/useradd. The following the output of the useradd -D command which list the default parameters when a new user is created:

```
sysadmin@localhost:~$ useradd -D
GROUP=100
HOME=/home
INACTIVE=-1
EXPIRE=
SHELL=/bin/sh
SKEL=/etc/skel
CREATE_MAIL_SPOOL=no
```

- You can execute the useradd account using the following options to control the outcome:

```
useradd  -s  <Shell>  -d <Home directory>  -m  -k <Skeleton directory>  -g <Group>  username
```

- The table on the next slide gives a detailed explanation of each option.

4.5.1 Managing User Accounts (Create) (Cont'd)

Option	Meaning
`-s`	User's default login shell. The default on most Linux systems is `/bin/bash`. Other popular alternatives are: `/bin/tsh`, `/bin/dash`, `/bin/zsh`, and `/bin/ksh`.
`-d`	The home directory for the new user. The default on most Linux systems is `/home/username`.
`-m`	Creates the home directory for the user if it does not exist.
`-k`	Copy initialization files from an alternative directory. The default is `/etc/skel`.
`-g`	Group name or number of the user. The default group varies depending on the Linux distribution.
`-N`	Avoids creation of a group with the same name as the user name. An alternative group should be specified with the `-g` option.

- The example below shows the execution of the useradd command with typical options. You will notice that the user home directory, shell, primary group are set by the useradd command options.

```
sysadmin@localhost:~$ sudo useradd -s /bin/bash -d /var/user1 -m -g 0 user1
sysadmin@localhost:~$ sudo passwd user1
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
sysadmin@localhost:~$ su - user1
Password:
user1@localhost:~$ pwd
/var/user1
user1@localhost:~$ id
uid=1002(user1) gid=0(root) groups=0(root)
user1@localhost:~$ echo $SHELL
/bin/bash
```

4.5.2 Managing User Accounts (Modify)

- After a user account has been created, an administrator may need to make some changes to the account parameters to accommodate a job role change, system changes or to simply update or correct existing user information.
- The usermod command is user by the root account to modify existing user account parameters such as home directory, login name, login shell, password aging information etc.
- For example, to change the location of the test_user account from /home/test_user to /home/users/test_user, you can use the following commands:

```
root@localhost:~# mkdir -p /home/users
root@localhost:~# usermod -d /home/users/test_user test_user
root@localhost:~# grep test_user /etc/passwd
test_user:x:1002:1002::/home/users/test_user:/bin/bash
```

- To lock and unlock a user account, we can use the -L and -U options respectively:

```
root@localhost:~# usermod -L test_user
root@localhost:~# passwd -S test_user
test_user L 12/15/2019 0 99999 7 -1
root@localhost:~# usermod -U test_user
root@localhost:~# passwd -S test_user
test_user P 12/15/2019 0 99999 7 -1
```

4.5.2 Managing User Accounts (Modify) (Cont'd)

- One of the most important use of the usermod command is to modify the group membership of a user account. To modify the primary group of the user, use the -g option while to modify the secondary groups of the user, use the -G option.

```
root@localhost:~# usermod -g root test_user
root@localhost:~# id test_user
uid=1002(test_user) gid=0(root) groups=0(root)
```

```
root@localhost:~# usermod -G sysadmin test_user
root@localhost:~# id test_user
uid=1002(test_user) gid=0(root) groups=0(root),1001(sysadmin)
```

- User the -a option to append new secondary groups for a user, else you will need to list all the secondary groups of the user if only the -G option is used. Notice in the first example below, newgrp replaces sysadmin group as the secondary group when only -G is used. When -a option is used in conjunction with -G, sysadmin group is now added to the list of secondary groups for test_user.
- Alternatively, we could also have used the following command to achieve the same outcome.

usermod -G newgrp,sysadmin test_user

```
root@localhost:~# usermod -G newgrp test_user
root@localhost:~# id test_user
uid=1002(test_user) gid=0(root) groups=0(root),1003(newgrp)
root@localhost:~# usermod -a -G sysadmin test_user
root@localhost:~# id test_user
uid=1002(test_user) gid=0(root) groups=0(root),1001(sysadmin),1003(newgrp)
```

4.5.3 Managing User Accounts (Delete)

- User deletion is accomplished by the userdel command. It will remove the user entry in /etc/passwd and /etc/shadow including any references to the user account in /etc/group file. However, the home directory and any files and directories created by the user is not removed. To delete the user account along with the user's home directory and mail spool, use the -r option.

```
root@localhost:~# useradd test_userdel
root@localhost:~# useradd -d /home/testuser2 -m testuser2
root@localhost:~# userdel testuser2
root@localhost:~# ls -ld /home/testuser2
drwxr-xr-x 2 1003 1005 4096 Dec 15 01:30 /home/testuser2
root@localhost:~# useradd -d /home/testuser3 -m testuser3
root@localhost:~# userdel -r testuser3
userdel: testuser3 mail spool (/var/mail/testuser3) not found
root@localhost:~# ls -ld /home/testuser3
ls: cannot access /home/testuser3: No such file or directory
```

- Note that to delete any additional directories or files created by deleted users, you will need to use the find command as follows:

```
root@localhost:~# find /var -nouser | xargs ls -l
-rw-r--r-- 1 1003 1005    0 Dec 15 01:37 /var/testuser4/testfile4

/var/testuser4:
total 0
-rw-r--r-- 1 1003 1005 0 Dec 15 01:37 testfile4
```

4.6 Updating User Passwords

- The passwd command is used to update a user's password. Users can only change their own passwords while the root user can update the password for any user.
- In the example below, the sysadmin account is prevented from attempting to update dev1's account while both the root user or the dev1 user can successfully update the password

```
sysadmin@localhost:~$ passwd dev1
passwd: You may not view or modify password information for dev1.
sysadmin@localhost:~$ su -
Password:
root@localhost:~# passwd dev1
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
root@localhost:~# su - dev1
dev1@localhost:~$ passwd
Changing password for dev1.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

4.6 Updating User Passwords (Cont'd)

- If the user wish to view status information about his password, they can use the -S option

```
dev1@localhost:~$ passwd -S dev1
dev1 P 12/14/2019 0 99999 7 -1
```

- The table explains each field of the output of the passwd -S command

Field	Example	Meaning
User Name	`sysadmin`	Name of the user
Password status	`P`	`P` means "usable password" `L` means "locked password" `NP` means "no password"
Last password change date	`03/01/2015`	Date when the password was last changed
Minimum	`0`	Minimum number of days that must pass before the current password can be changed by the user
Maximum	`99999`	Maximum number of days remaining for the password to expire
Warn	`7`	Number of days prior to password expiry that the user is warned
Inactive	`-1`	Number of days after password expiry that the user account remains active

- The root user can force a password change by the user during the next login by executing the passwd with the -e option to expire the account.

```
root@localhost:~# passwd -e dev1
passwd: password expiry information changed.
```

4.7.1 Managing Groups (Create)

- The groupadd command is used to create a new group. For example, to add a new group named programmers, execute the following command as root:

```
root@localhost:~# groupadd programmers
root@localhost:~# tail -1 /etc/group
programmers:x:1003
```

- By default, the groupadd command automatically assigns a GID to the new group using the next available GID. To create a group with a specific GID, use the –g option:

```
root@localhost:~# groupadd programmers -g 1980
root@localhost:~# tail -1 /etc/group
programmers:x:1980
```

4.7.2 Managing Groups (Modify)

- You can use the groupmod command with the -g option to change the group id of the group.

```
root@localhost:~# groupmod programmers -g 1990
root@localhost:~# tail -1 /etc/group
programmers:x:1990
```

- Similar to the /etc/shadow file, the /etc/gshadow file is used to store group password information and other security related group data. If a group is added, deleted or modified, the gshadow file is also updated. Each record in the /etc/gshadow file contains the following four fields:

```
group_name:encrypted_password:group_administrators:members
```

Field	Example	Significance
Group Name	`programmers`	Valid existing group name
Encrypted Password	`$1$nfffgggc$pGt#4`	Used when a user who is not a member of the group wants to gain permission to this group. An exclamation mark at the beginning of the password indicates it is locked.
Group Admininstrators	`useradmin`	Comma-separated list of group administrators
Members	`sarah,mickey,dinesh`	Comma-separated list of group users that will not be prompted for a password

- The gpasswd command is used by the root user to update the group information in the /etc/gshadow file.

4.7.3 Managing Groups (Delete)

- The groupdel command is used to delete a group that is no longer needed.

```
root@localhost:~# groupdel programmers
```

- If the group is currently used as a primary group for some user account, an error will occur and the group cannot be removed until the user has been reassigned to another group.

```
root@localhost:~# groupdel sysadmin
groupdel:  cannot remove the primary group of user 'sysadmin'
```

4.8 Password Policy

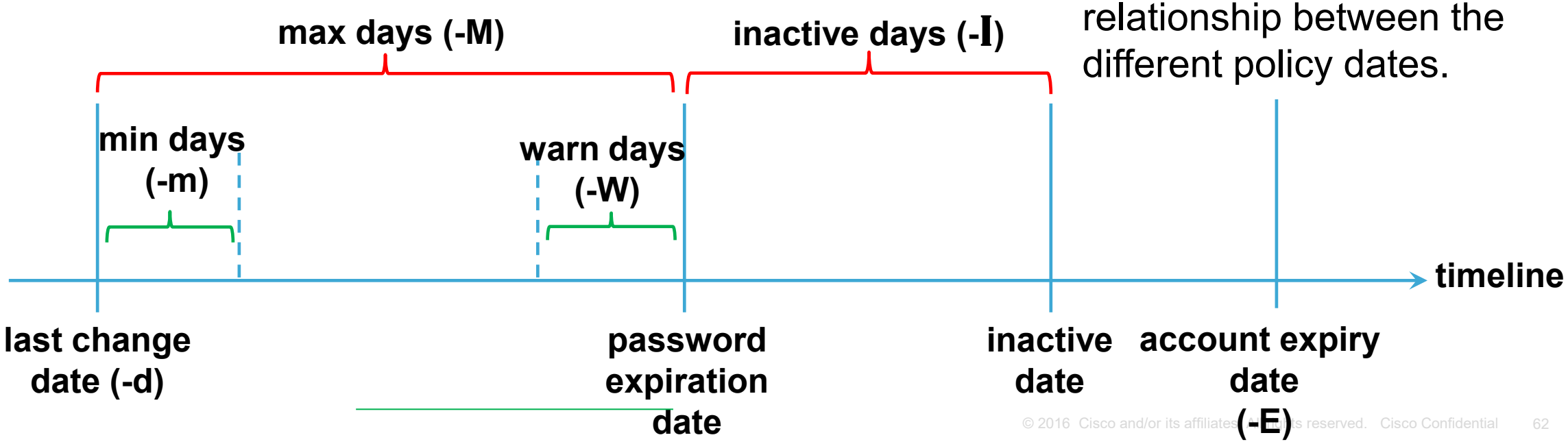
- The chage command is used to update information related to password expiration. Using this command, the administrator can enforce a password changing and expiry policy for specific user accounts.
- Note that this command only affects specific current users and not new users created on the system. To determine the password expiry and change frequency policy, use the PASS_MAX_DAYS, PASS_MIN_DAYS and other settings in the /etc/login.defs file.
- The chage command can be used to change the maximum and minimum number of days between password expiry by using the -M and -m options respectively. Use the -l option to list the current policy.

```
root@localhost:~# chage -l dev1
Last password change                                    : Dec 14, 2019
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
root@localhost:~# chage -M 90 -m 2 dev1
root@localhost:~# chage -l dev1
Last password change                                    : Dec 14, 2019
Password expires                                        : Mar 13, 2020
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 2
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 7
```

4.8 Password Policy (Cont'd)

```
root@localhost:~# chage -W 7 -I 10 -E "2020-07-01" dev1
root@localhost:~# chage -l dev1
Last password change                                    : Dec 14, 2019
Password expires                                        : Mar 13, 2020
Password inactive                                       : Mar 23, 2020
Account expires                                         : Jul 01, 2020
Minimum number of days between password change          : 2
Maximum number of days between password change          : 90
Number of days of warning before password expires       : 7
```

- Other policies that can be set include the number of days prior to password expiry to warn the user (-w option), the number of grace days to allow the user to change password after the password has expired and the account (not password) expiry date (after which the account will be locked).
  - The chart below depicts the relationship between the different policy dates.



4.9 Restricting Access

- The root user can lock and unlock a user's account using the passwd -l <username> and passwd -u <username> respectively.
- In the example on the right, notice that the sysadmin account password status changes from P to L and then back to P.
- The root user can also remove the password for an account using the passwd -d option
- In the example below, the dev1 account passwd is removed. Notice the hash password stored in the /etc/shadow file has disappeared after the password deletion.

```
root@localhost:~# passwd -S sysadmin
sysadmin P 03/14/2016 0 99999 7 -1
root@localhost:~# passwd -l sysadmin
passwd: password expiry information changed.
root@localhost:~# passwd -S sysadmin
sysadmin L 03/14/2016 0 99999 7 -1
root@localhost:~# passwd -u sysadmin
passwd: password expiry information changed.
root@localhost:~# passwd -S sysadmin
sysadmin P 03/14/2016 0 99999 7 -1
```

```
root@localhost:~# grep dev1 /etc/shadow
dev1:$6$oQZ.OS65$2QWfvoXKr0UPQ4autmZxCmJ0EFFyxC
PY9sW/RwkKgPcOfWA8wxxF0:18244:0:99999:7:::
root@localhost:~# passwd -d dev1
passwd: password expiry information changed.
root@localhost:~# grep dev1 /etc/shadow
dev1::18244:0:99999:7:::
root@localhost:~#
```

4.10 Miscellaneous (User Private Groups)

- By default, whenever a new account is created using the useradd command, a group with the same name as the account being created is also created.

```
sysadmin@localhost:~$ sudo useradd test_user1
sysadmin@localhost:~$ tail -1 /etc/group
test_user1:x:1003:
sysadmin@localhost:~$ tail -1 /etc/passwd
test_user1:x:1003:1003::/home/test_user1:
sysadmin@localhost:~$ sudo id test_user1
uid=1003(test_user1) gid=1003(test_user1) groups=1003(test_user1)
```

- You will notice that the test_user1 group is automatically created and also assigned as the primary group for the test_user1 account.

- The User Private Group behavior is overridden when the useradd command is run with -N or -g options.
- Notice that by using the -g option, the test_user2 group is not created. The id command shows that the primary group of test_user2 is test_user1

```
sysadmin@localhost:~$ sudo useradd -g test_user1 test_user2
sysadmin@localhost:~$ grep test_user2 /etc/group
sysadmin@localhost:~$
sysadmin@localhost:~$ id test_user2
uid=1004(test_user2) gid=1003(test_user1) groups=1003(test_user1)
```

4.10 Miscellaneous (Template Initialization Files)

- The /etc/skel directory contains the default set of “skeleton” (template) files and directories, which are copied to the home directory of a user that is created with the useradd command. This provides a uniform directory structure for all new users of the system.

Important notes:

- The /etc/skel directory is only used to propagate accounts when an account is created. If you add files to the /etc/skel directory, then these files are not automatically added to existing user’s home directories
- After the files are copied into the user’s home directories, the ownership of these new files are changed to the new user account. This means that a user can modify the contents of these files, since they are owned by the user. Additionally, nothing prevents the user from deleting these files, since a user can delete any file in their own home directory.
- Some of the commonly found files in this directory are:
  - .bashrc - contains alias definitions, enables shell features, etc.
  - .profile - contains variable definitions and one-time-only commands to set the user’s environment
  - .bash_logout - contains actions to take when logging out (i.e. clear the screen, etc.)

4.10 Miscellaneous (Template Initialization Files) (Cont'd)

- The example below shows that test_user4 is created with template files .bashrc, .profile and .selected_editor
- Notice that after the file .mozilla is created in the /etc/skel directory, test_user5 now has .mozilla in the home directory while test_user4 do not.
- It is also possible to create different skel directories for different groups of user accounts. You may create a different skel directory for developers, sales and admin staff such that they have different umask, path and aliases.
- The example on the right creates a developer account dev1 using a different skel directory

```
sysadmin@localhost:~$ sudo ls -a /etc/skel
.  ..  .bash_logout  .bashrc  .profile  .selected_editor
sysadmin@localhost:~$ sudo useradd -d /home/test_user4 -m test_user4
sysadmin@localhost:~$ sudo ls -a /home/test_user4
.  ..  .bash_logout  .bashrc  .profile  .selected_editor
sysadmin@localhost:~$ sudo touch /etc/skel/.mozilla
sysadmin@localhost:~$ sudo useradd -d /home/test_user5 -m test_user5
sysadmin@localhost:~$ sudo ls -a /home/test_user5
.  ..  .bash_logout  .bashrc  .mozilla  .profile  .selected_editor
sysadmin@localhost:~$ sudo ls -a /home/test_user4
.  ..  .bash_logout  .bashrc  .profile  .selected_editor
```

```
sysadmin@localhost:~$ sudo ls -a /etc/skel_dev
.  ..  .bash_logout  .bashrc  .developer  .profile
sysadmin@localhost:~$ sudo useradd -d /home/dev1 -m -k /etc/skel_dev/ dev1
sysadmin@localhost:~$ sudo su - dev1
dev1@localhost:~$ pwd
/home/dev1
dev1@localhost:~$ ls -a
.  ..  .bash_logout  .bashrc  .developer  .profile
```

Lab

RH124 Chapter 5.7 Lab Creating, Viewing and Editing Text Files

RH124 Chapter 6.11 Lab Managing Local Users and Groups

Practice Exercises

1 Redirection & Pipes

•Use > to redirect the output of ls to files.txt.
•Append system uptime to files.txt using >>.
•Redirect standard error from ls /invalid_path to errors.txt.
•Use /dev/null to discard error messages.
•Pipe ls /etc into grep to filter files ending in .conf.

2 Working with vi and vim

•Open notes.txt in vi. Enter insert mode and type a sentence.
•Save and exit using the proper vi command.
•Try out vimtutor

3 Shell Variables & Environment

•Create a variable myname with your name as its value.
•Display the variable's value using echo.
•Convert myname to an environment variable.
•Display the contents of the PATH variable.
•Unset the myname variable and check if it still exists.

4 User & Group Management

•Create a user student_test with home directory /home/student_test.
•Set a password for student_test.
•Create a group testers and add student_test to it.
•Change student_test's primary group to testers.
•Lock student_test's account, then unlock it.